

DAY 7- PROGRAMMING

Name: Y.Thanushma

Registration Number:192425355

Course Code:CSA0617-Design&Analysis of Algorithms

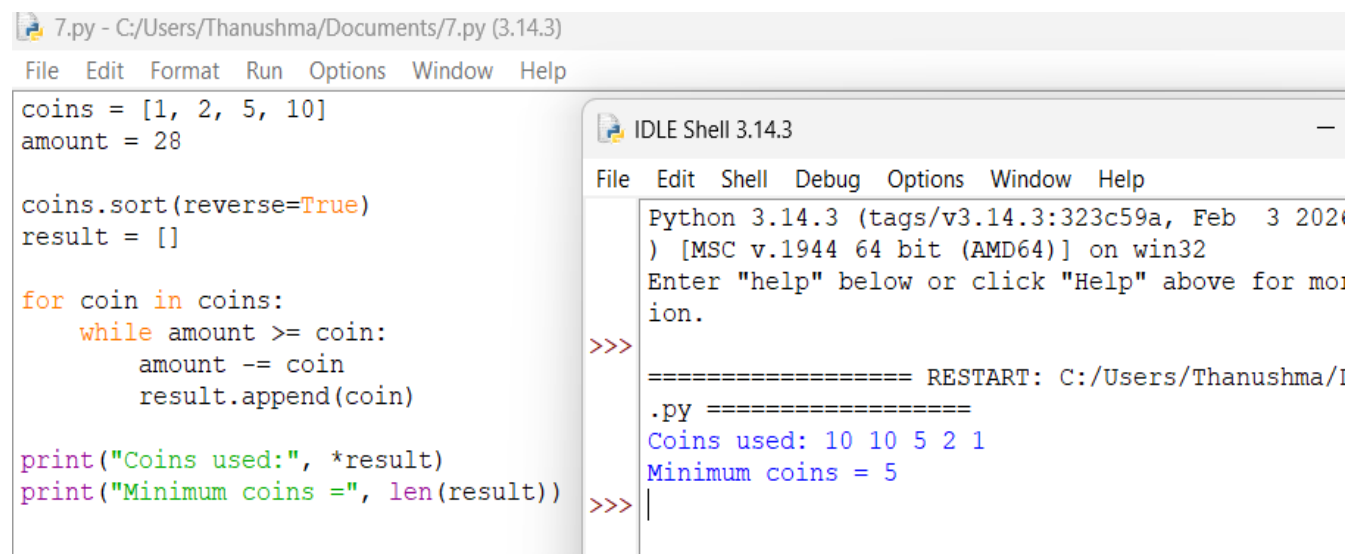
1. Coin Change – Supermarket Billing

Aim:To find the minimum number of coins required to return a balance amount using the Greedy Coin Change algorithm.

Algorithm

1. Read the coin denominations and amount.
2. Sort the coins in descending order.
3. Select the largest coin that does not exceed the remaining amount.
4. Subtract the selected coin from the amount.
5. Repeat until the amount becomes zero.
6. Display the selected coins and minimum number of coins.

Execution:



```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help

coins = [1, 2, 5, 10]
amount = 28

coins.sort(reverse=True)
result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins used:", *result)
print("Minimum coins =", len(result))

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2024) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Thanushma/1
.py =====
Coins used: 10 10 5 2 1
Minimum coins = 5
>>>
```

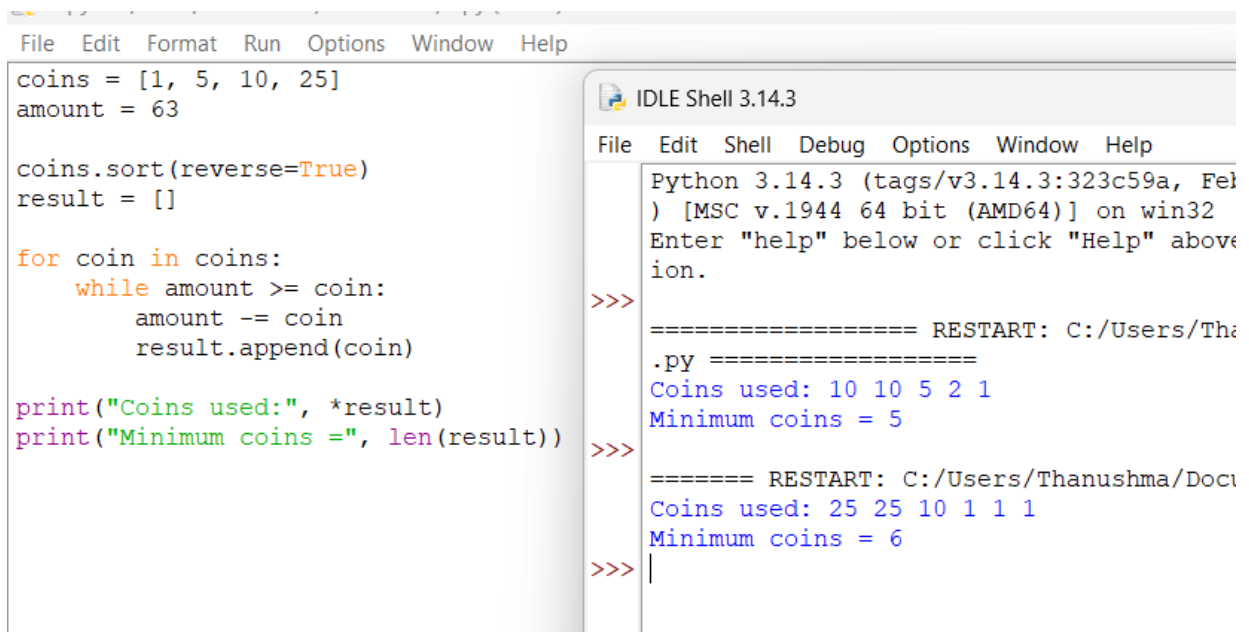
2. Coin Change – Parking Meter

Aim: To determine the minimum number of coins needed to make the given parking fee using the Greedy method.

Algorithm

1. Read the denominations and amount.
2. Sort denominations in descending order.
3. Choose the largest possible coin repeatedly.
4. Subtract it from the remaining amount.
5. Continue until the amount becomes zero.
6. Count and display the coins.

Execution:



```
File Edit Format Run Options Window Help
coins = [1, 5, 10, 25]
amount = 63

coins.sort(reverse=True)
result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins used:", *result)
print("Minimum coins =", len(result))

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 19 2024) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above the shell.

>>>
===== RESTART: C:/Users/Thanushma/Python314/Python314 Shell
Coins used: 10 10 5 2 1
Minimum coins = 5
>>>
===== RESTART: C:/Users/Thanushma/Python314/Python314 Shell
Coins used: 25 25 10 1 1 1
Minimum coins = 6
>>>
```

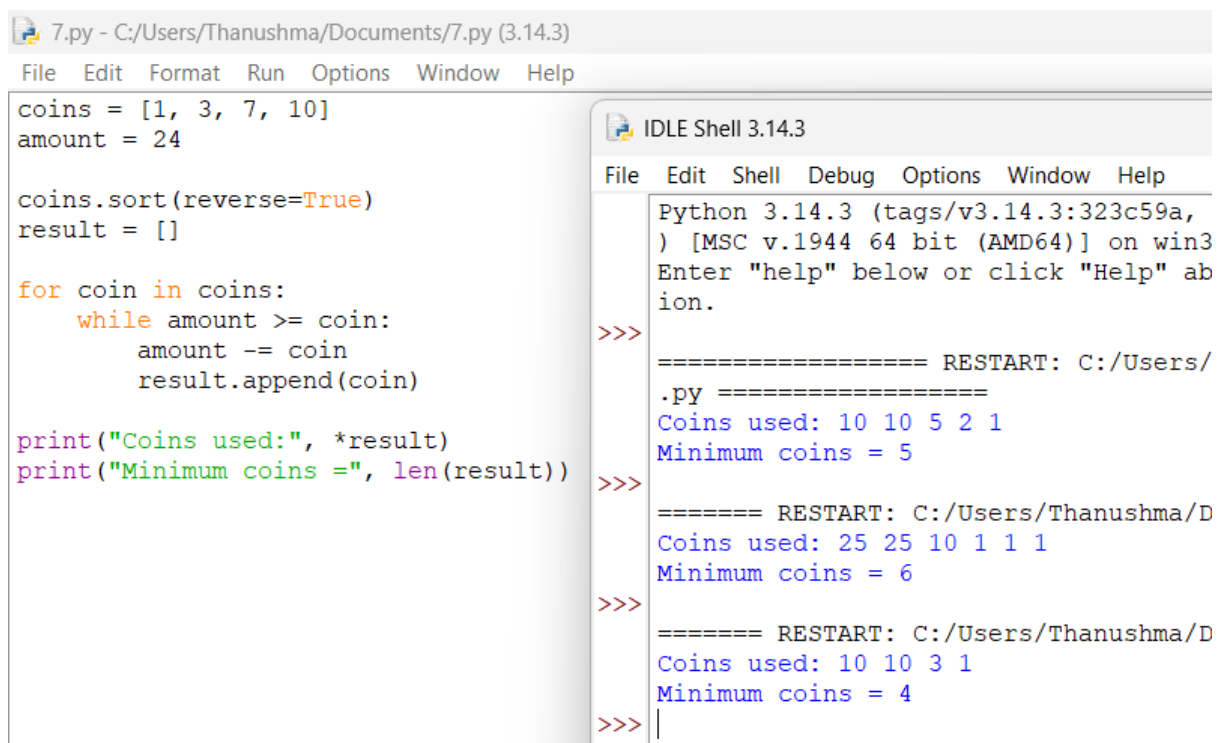
3. Coin Change – Ticket Counter

Aim: To find the minimum number of coins required to return change using available denominations.

Algorithm

1. Read coins and amount.
2. Sort coins in descending order.
3. Select the largest possible coin.
4. Subtract it from the amount.
5. Repeat until the amount is zero.
6. Display the coins and their count.

Execution:



```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help
coins = [1, 3, 7, 10]
amount = 24

coins.sort(reverse=True)
result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins used:", *result)
print("Minimum coins =", len(result))

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, ) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" button.

>>>
===== RESTART: C:/Users/Thanushma/D
.py =====
Coins used: 10 10 5 2 1
Minimum coins = 5
>>>
===== RESTART: C:/Users/Thanushma/D
Coins used: 25 25 10 1 1 1
Minimum coins = 6
>>>
===== RESTART: C:/Users/Thanushma/D
Coins used: 10 10 3 1
Minimum coins = 4
>>> |
```

4. Fractional Knapsack – Food Delivery Truck

Aim: To maximize the total profit of food packets that can be carried using the Fractional Knapsack algorithm.

Algorithm

1. Calculate profit-to-weight ratio for every item.

2. Sort items in decreasing order of ratio.
3. Select the complete item if it fits.
4. Otherwise, take the required fraction of the item.
5. Continue until the capacity becomes zero.
6. Display the maximum profit.

Execution:

<pre> 7.py - C:/Users/Thanushma/Documents/7.py (3.14.3) File Edit Format Run Options Window Help weights = [10, 20, 30] profits = [60, 100, 120] capacity = 50 items = [(profits[i] / weights[i], weights[i], for i in range(len(weights))]) items.sort(reverse=True) total_profit = 0 for ratio, weight, profit in items: if capacity >= weight: capacity -= weight total_profit += profit else: total_profit += ratio * capacity break print("Maximum profit =", total_profit) </pre>	<pre> IDLE Shell 3.14.3 File Edit Shell Debug Options Window Help Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 16:04:56) [MSC v.1944 64 bit (AMD64)] on wi Enter "help" below or click "Help" above fo information. >>> ===== RESTART: C:/Users/Thanus uments/7.py ===== Coins used: 10 10 5 2 1 Minimum coins = 5 >>> ===== RESTART: C:/Users/Thanushma/Documer ===== Coins used: 25 25 10 1 1 1 Minimum coins = 6 >>> ===== RESTART: C:/Users/Thanushma/Documer ===== Coins used: 10 10 3 1 Minimum coins = 4 >>> ===== RESTART: C:/Users/Thanushma/Documer ===== Maximum profit = 240.0 >>> </pre>
--	---

5. Fractional Knapsack – Gold Transport

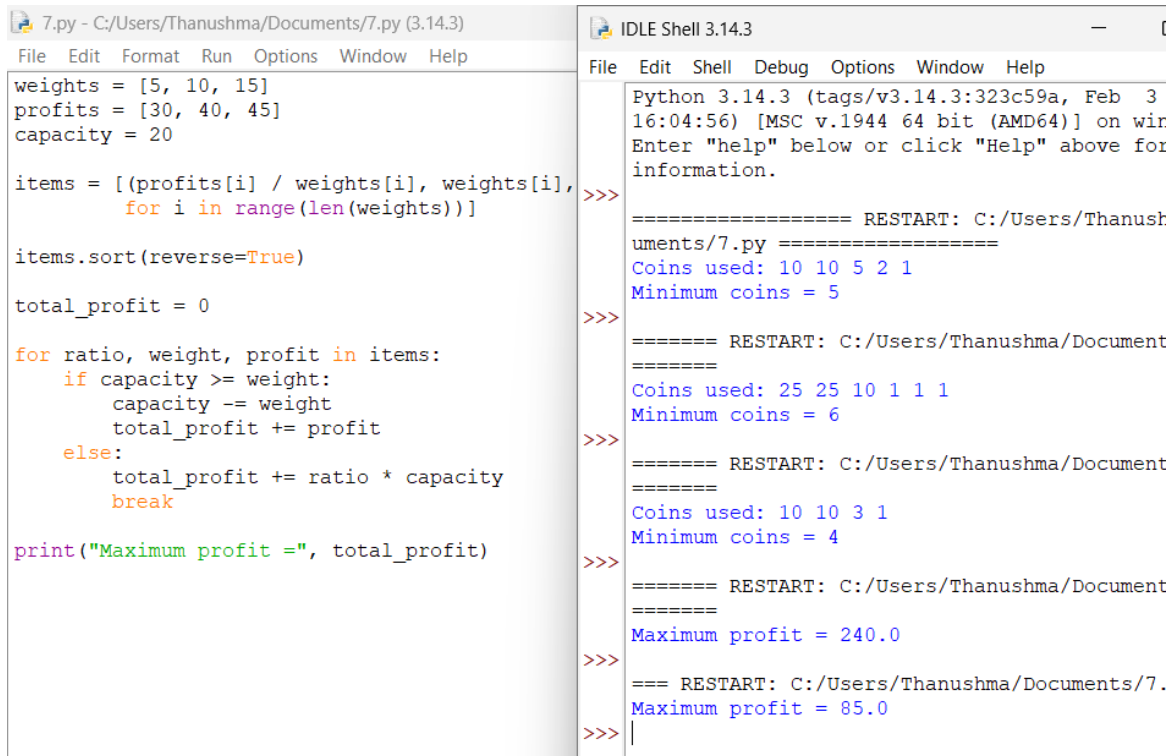
Aim: To maximize the value of gold transported within the given weight capacity.

Algorithm

1. Calculate profit/weight ratio.
2. Sort items by decreasing ratio.
3. Select the item with the highest ratio first.
4. Take complete items while possible.
5. Take a fraction of the next item if necessary.

6. Calculate the maximum profit.

Execution:



```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help
weights = [5, 10, 15]
profits = [30, 40, 45]
capacity = 20

items = [(profits[i] / weights[i], weights[i],
          for i in range(len(weights))])

items.sort(reverse=True)

total_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        total_profit += profit
    else:
        total_profit += ratio * capacity
        break

print("Maximum profit =", total_profit)

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3
16:04:56) [MSC v.1944 64 bit (AMD64)] on win
Enter "help" below or click "Help" above for
information.

===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Coins used: 10 10 5 2 1
Minimum coins = 5

>>>

===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Coins used: 25 25 10 1 1 1
Minimum coins = 6

>>>

===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Coins used: 10 10 3 1
Minimum coins = 4

>>>

===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Maximum profit = 240.0

>>>

===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Maximum profit = 85.0

>>>
```

6. Fractional Knapsack – Relief Material Loading

Aim: To maximize the importance value of relief materials loaded within a limited capacity.

Algorithm

1. Calculate the profit-to-weight ratio.
2. Sort items according to decreasing ratio.
3. Select the highest-ratio items first.
4. Take complete items whenever possible.
5. Take a fraction when the remaining capacity is insufficient.
6. Display maximum profit.

Execution:

```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help
weights = [4, 8, 2, 6]
profits = [20, 40, 14, 30]
capacity = 10

items = [(profits[i] / weights[i], weights[i],
          for i in range(len(weights)))]

items.sort(reverse=True)

total_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        total_profit += profit
    else:
        total_profit += ratio * capacity
        break

print("Maximum profit =", total_profit)

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, 16:04:56) [MSC v.1944 64 bit (AMD64)]
Enter "help" below or click "Help" at
information.

>>> ===== RESTART: C:/Users.
uments/7.py =====
Coins used: 10 10 5 2 1
Minimum coins = 5

>>> ===== RESTART: C:/Users/Thanushma/.
=====
Coins used: 25 25 10 1 1 1
Minimum coins = 6

>>> ===== RESTART: C:/Users/Thanushma/.
=====
Coins used: 10 10 3 1
Minimum coins = 4

>>> ===== RESTART: C:/Users/Thanushma/.
=====
Maximum profit = 240.0

>>> === RESTART: C:/Users/Thanushma/Docu
Maximum profit = 85.0

>>> === RESTART: C:/Users/Thanushma/Docu
Maximum profit = 54.0

...

```

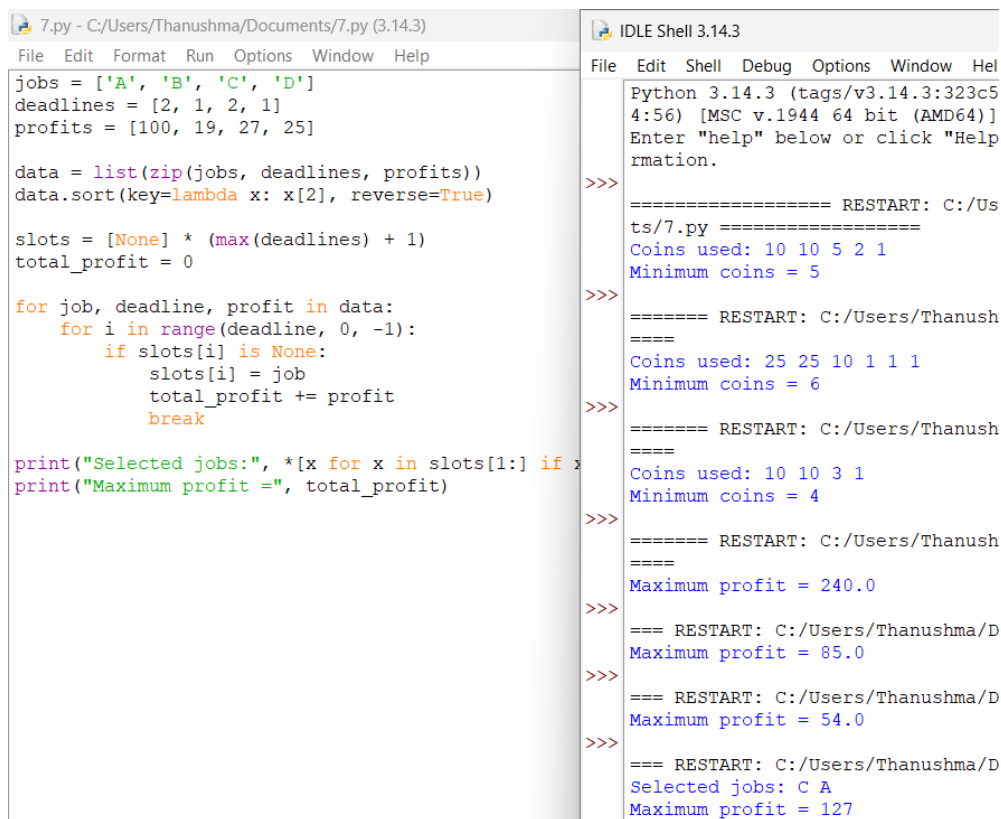
7. Job Sequencing – Software Company Tasks

Aim: To schedule jobs before their deadlines to obtain maximum profit using the Job Sequencing with Deadlines algorithm.

Algorithm

1. Store job name, deadline, and profit.
2. Sort jobs in decreasing order of profit.
3. Create empty time slots.
4. For each job, find the latest available slot before its deadline.
5. Schedule the job if a slot is available.
6. Calculate and display maximum profit.

Execution:



```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help

jobs = ['A', 'B', 'C', 'D']
deadlines = [2, 1, 2, 1]
profits = [100, 19, 27, 25]

data = list(zip(jobs, deadlines, profits))
data.sort(key=lambda x: x[2], reverse=True)

slots = [None] * (max(deadlines) + 1)
total_profit = 0

for job, deadline, profit in data:
    for i in range(deadline, 0, -1):
        if slots[i] is None:
            slots[i] = job
            total_profit += profit
            break

print("Selected jobs:", *[x for x in slots[1:] if x])
print("Maximum profit =", total_profit)
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Hel
Python 3.14.3 (tags/v3.14.3:323c5
4:56) [MSC v.1944 64 bit (AMD64)]
Enter "help" below or click "Help
rmation.

>>>
===== RESTART: C:/Us
ts/7.py =====
Coins used: 10 10 5 2 1
Minimum coins = 5

>>>
===== RESTART: C:/Users/Thanush
=====
Coins used: 25 25 10 1 1 1
Minimum coins = 6

>>>
===== RESTART: C:/Users/Thanush
=====
Coins used: 10 10 3 1
Minimum coins = 4

>>>
===== RESTART: C:/Users/Thanush
=====
Maximum profit = 240.0

>>>
=== RESTART: C:/Users/Thanushma/D
Maximum profit = 85.0

>>>
=== RESTART: C:/Users/Thanushma/D
Maximum profit = 54.0

>>>
=== RESTART: C:/Users/Thanushma/D
Selected jobs: C A
Maximum profit = 127
```

8. Job Sequencing – Exam Paper Evaluation

Aim: To schedule paper evaluation jobs before their deadlines while maximizing the total reward.

Algorithm

1. Read jobs, deadlines, and profits.
2. Sort jobs by decreasing profit.
3. Create available time slots.
4. Assign each job to the latest available slot before its deadline.
5. Skip a job if no slot is available.
6. Display selected jobs and maximum profit.

Execution:

```
File Edit Format Run Options Window Help
jobs = ['J1', 'J2', 'J3', 'J4', 'J5']
deadlines = [2, 1, 2, 1, 3]
profits = [20, 15, 10, 5, 1]

data = list(zip(jobs, deadlines, profits))
data.sort(key=lambda x: x[2], reverse=True)

slots = [None] * (max(deadlines) + 1)
total_profit = 0

for job, deadline, profit in data:
    for i in range(deadline, 0, -1):
        if slots[i] is None:
            slots[i] = job
            total_profit += profit
            break

print("Selected jobs:", *[x for x in slots[1:] if x])
print("Maximum profit =", total_profit)
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2
AMD64)] on win32
Enter "help" below or click "Help" above for
>>>
===== RESTART: C:/Users/Thanushr
Selected jobs: J2 J1 J5
Maximum profit = 36
>>> |
```

9. Job Sequencing – Printing Press Orders

Aim: To schedule printing orders before their deadlines to maximize total profit.

Algorithm

1. Read job details. Sort jobs by decreasing profit.
2. Create time slots according to the maximum deadline.
3. Place each job in the latest free slot. Calculate the total profit.
4. Display selected jobs.

Execution:

```
File Edit Format Run Options Window Help
jobs = ['A', 'B', 'C', 'D', 'E']
deadlines = [2, 1, 2, 1, 3]
profits = [100, 50, 10, 20, 30]

data = list(zip(jobs, deadlines, profits))
data.sort(key=lambda x: x[2], reverse=True)

slots = [None] * (max(deadlines) + 1)
total_profit = 0

for job, deadline, profit in data:
    for i in range(deadline, 0, -1):
        if slots[i] is None:
            slots[i] = job
            total_profit += profit
            break

print("Selected jobs:", *[x for x in slots[1:] if x])
print("Maximum profit =", total_profit)
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, 1
AMD64)] on win32
Enter "help" below or click "Help" abo
>>>
===== RESTART: C:/Users/.
Selected jobs: J2 J1 J5
Maximum profit = 36
>>>
===== RESTART: C:/Users/.
Selected jobs: B A E
Maximum profit = 180
>>>
```

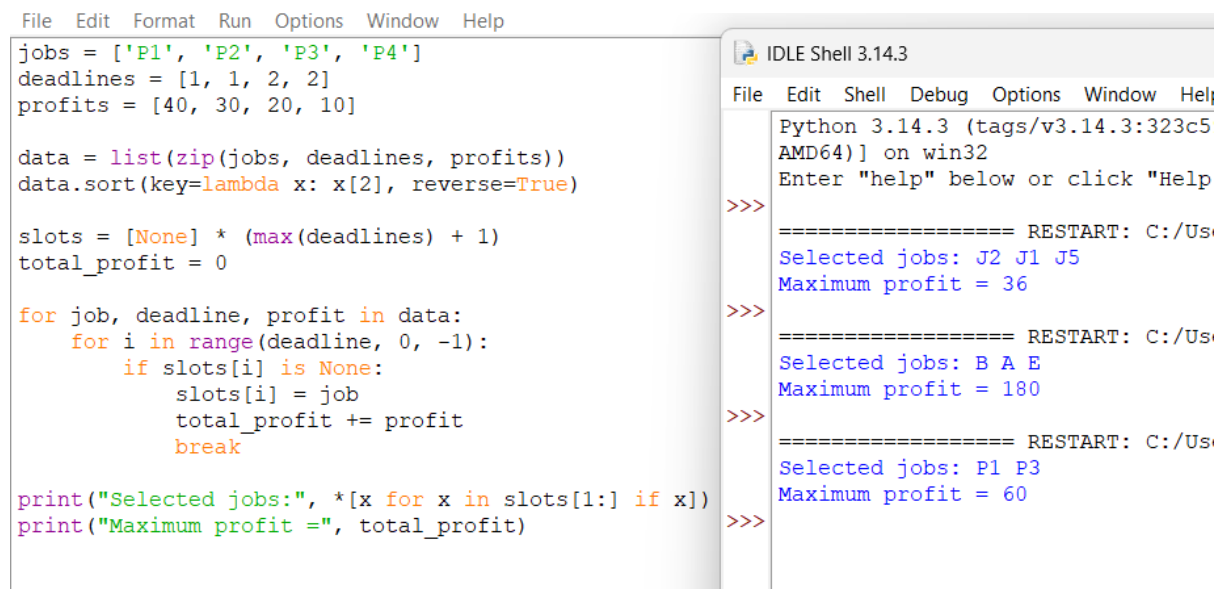

10. Job Sequencing – Freelance Projects

Aim: To schedule freelance projects before deadlines to maximize total income.

Algorithm

1. Read project names, deadlines, and profits.
2. Sort projects in decreasing order of profit.
3. Create available time slots.
4. Assign each project to the latest free slot.
5. Calculate the total income.
6. Display selected projects.

Execution:



```
File Edit Format Run Options Window Help
jobs = ['P1', 'P2', 'P3', 'P4']
deadlines = [1, 1, 2, 2]
profits = [40, 30, 20, 10]

data = list(zip(jobs, deadlines, profits))
data.sort(key=lambda x: x[2], reverse=True)

slots = [None] * (max(deadlines) + 1)
total_profit = 0

for job, deadline, profit in data:
    for i in range(deadline, 0, -1):
        if slots[i] is None:
            slots[i] = job
            total_profit += profit
            break

print("Selected jobs:", *[x for x in slots[1:] if x])
print("Maximum profit =", total_profit)
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c5 AMD64) on win32
Enter "help" below or click "Help"

>>> ===== RESTART: C:/Us
Selected jobs: J2 J1 J5
Maximum profit = 36

>>> ===== RESTART: C:/Us
Selected jobs: B A E
Maximum profit = 180

>>> ===== RESTART: C:/Us
Selected jobs: P1 P3
Maximum profit = 60

>>>
```

11. Huffman Coding – Text Compression

Aim: To generate Huffman codes for characters based on their frequencies and achieve efficient text compression.

Algorithm

1. Create a node for every character.
2. Insert all nodes into a min-priority queue.
3. Remove the two nodes with the lowest frequencies.
4. Create a new node with their combined frequency.
5. Insert the new node back into the queue.

6. Repeat until one node remains.
7. Traverse the Huffman tree and assign 0 to the left branch and 1 to the right branch.
8. Display the codes.

Execution:

```

7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help

import heapq

chars = ['a', 'b', 'c', 'd', 'e', 'f']
freq = [5, 9, 12, 13, 16, 45]

heap = [[freq[i], chars[i]] for i in range(len(chars))]
heapq.heapify(heap)

while len(heap) > 1:
    f1, c1 = heapq.heappop(heap)
    f2, c2 = heapq.heappop(heap)

    for item in heap:
        pass

    heapq.heappush(heap, [f1 + f2, (c1, c2)])

codes = {}

def generate(node, code=""):
    if isinstance(node, str):
        codes[node] = code
        return

    generate(node[0], code + "0")
    generate(node[1], code + "1")

generate(heap[0][1])

print("Huffman Codes:")
for ch in chars:
    print(ch + ":", codes[ch])

```

```

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:
AMD64)] on win32
Enter "help" below or click

>>>
===== RESTART:
Selected jobs: J2 J1 J5
Maximum profit = 36
>>>
===== RESTART:
Selected jobs: B A E
Maximum profit = 180
>>>
===== RESTART:
Selected jobs: P1 P3
Maximum profit = 60
>>>
===== RESTART:
Huffman Codes:
a: 1100
b: 1101
c: 100
d: 101
e: 111
f: 0
>>>

```

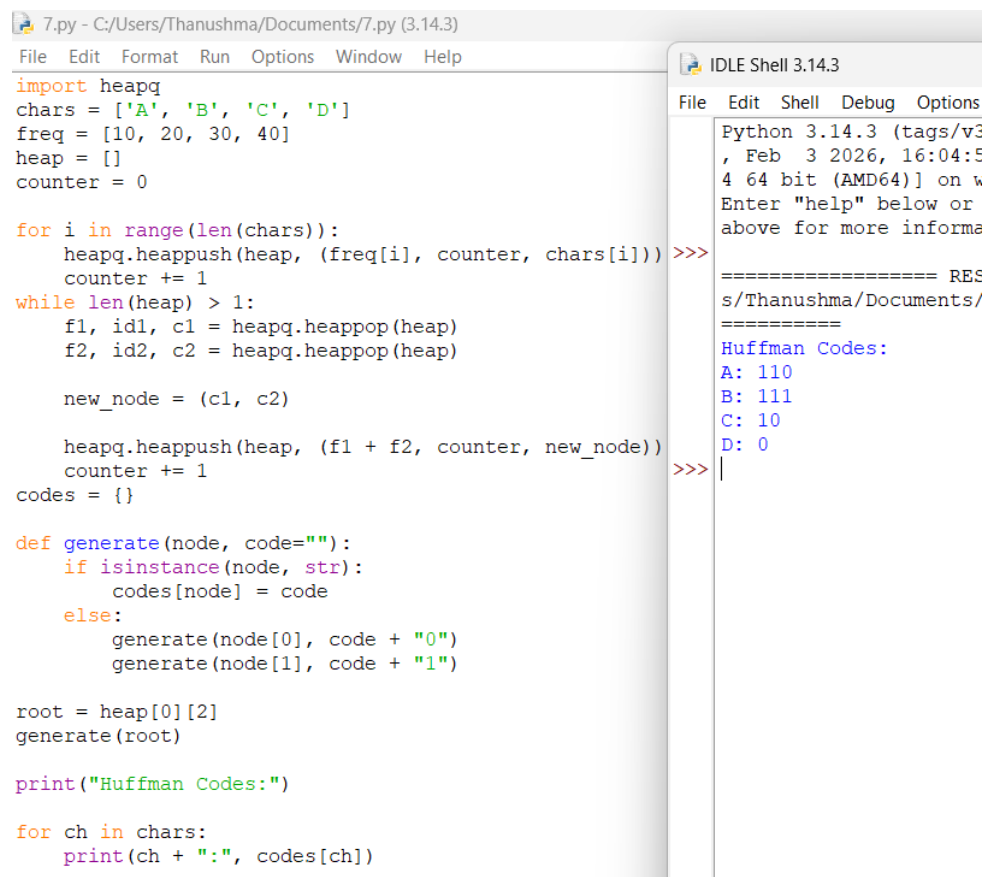
12. Huffman Coding – Message Encoding

Aim: To generate Huffman codes for frequently used message symbols to reduce the number of bits required for encoding.

Algorithm

1. Create nodes for all characters.
2. Insert nodes into a min-heap.
3. Remove the two nodes with minimum frequencies.
4. Combine them into a new node.
5. Insert the new node into the heap.
6. Repeat until one tree remains.
7. Traverse the tree and assign binary codes.
8. Display Huffman codes.

Execution:

The image shows a screenshot of the Python IDLE Shell interface. On the left, a Python script is being executed. The script defines a list of characters 'A', 'B', 'C', 'D' with frequencies 10, 20, 30, and 40 respectively. It uses a min-heap to build a Huffman tree. The script includes a 'generate' function to traverse the tree and produce binary codes. The output shows the Huffman codes: A: 110, B: 111, C: 10, and D: 0. On the right, the IDLE Shell window shows the execution output, including the Python version (3.14.3), the file path, and the resulting Huffman codes.

```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help

import heapq
chars = ['A', 'B', 'C', 'D']
freq = [10, 20, 30, 40]
heap = []
counter = 0

for i in range(len(chars)):
    heapq.heappush(heap, (freq[i], counter, chars[i]))
    counter += 1
while len(heap) > 1:
    f1, id1, c1 = heapq.heappop(heap)
    f2, id2, c2 = heapq.heappop(heap)

    new_node = (c1, c2)

    heapq.heappush(heap, (f1 + f2, counter, new_node))
    counter += 1
codes = {}

def generate(node, code=""):
    if isinstance(node, str):
        codes[node] = code
    else:
        generate(node[0], code + "0")
        generate(node[1], code + "1")

root = heap[0][2]
generate(root)

print("Huffman Codes:")

for ch in chars:
    print(ch + ":", codes[ch])

IDLE Shell 3.14.3
File Edit Shell Debug Options

Python 3.14.3 (tags/v3.14.3:255e265, Feb 3 2026, 16:04:54) 64 bit (AMD64) on win32
Enter "help()" below or above for more information.

===== RESULTS =====
Huffman Codes:
A: 110
B: 111
C: 10
D: 0
```

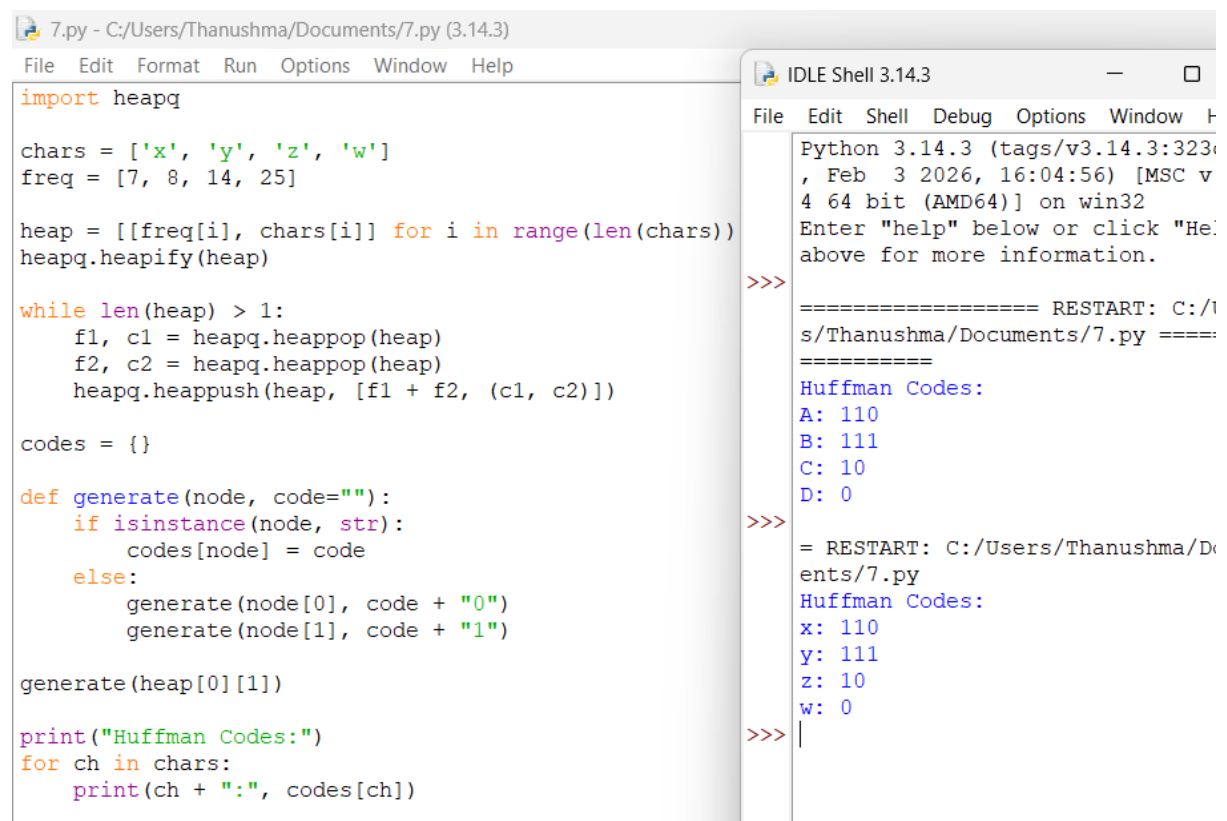
13. Huffman Coding – Sensor Data Compression

Aim: To generate Huffman codes for IoT sensor symbols and minimize the average number of bits required for data transmission.

Algorithm

1. Create a node for every sensor symbol.
2. Insert all nodes into a min-heap.
3. Select the two lowest-frequency nodes.
4. Combine their frequencies.
5. Insert the combined node back.
6. Repeat until one tree remains.
7. Traverse the tree to generate binary codes.
8. Display the codes.

Execution:



```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help

import heapq

chars = ['x', 'y', 'z', 'w']
freq = [7, 8, 14, 25]

heap = [[freq[i], chars[i]] for i in range(len(chars))]
heapq.heapify(heap)

while len(heap) > 1:
    f1, c1 = heapq.heappop(heap)
    f2, c2 = heapq.heappop(heap)
    heapq.heappush(heap, [f1 + f2, (c1, c2)])

codes = {}

def generate(node, code=""):
    if isinstance(node, str):
        codes[node] = code
    else:
        generate(node[0], code + "0")
        generate(node[1], code + "1")

generate(heap[0][1])

print("Huffman Codes:")
for ch in chars:
    print(ch + ":", codes[ch])

IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help

Python 3.14.3 (tags/v3.14.3:3230, Feb 3 2026, 16:04:56) [MSC v
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help"
above for more information.

>>>
===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Huffman Codes:
A: 110
B: 111
C: 10
D: 0
>>>
===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Huffman Codes:
x: 110
y: 111
z: 10
w: 0
>>>
```

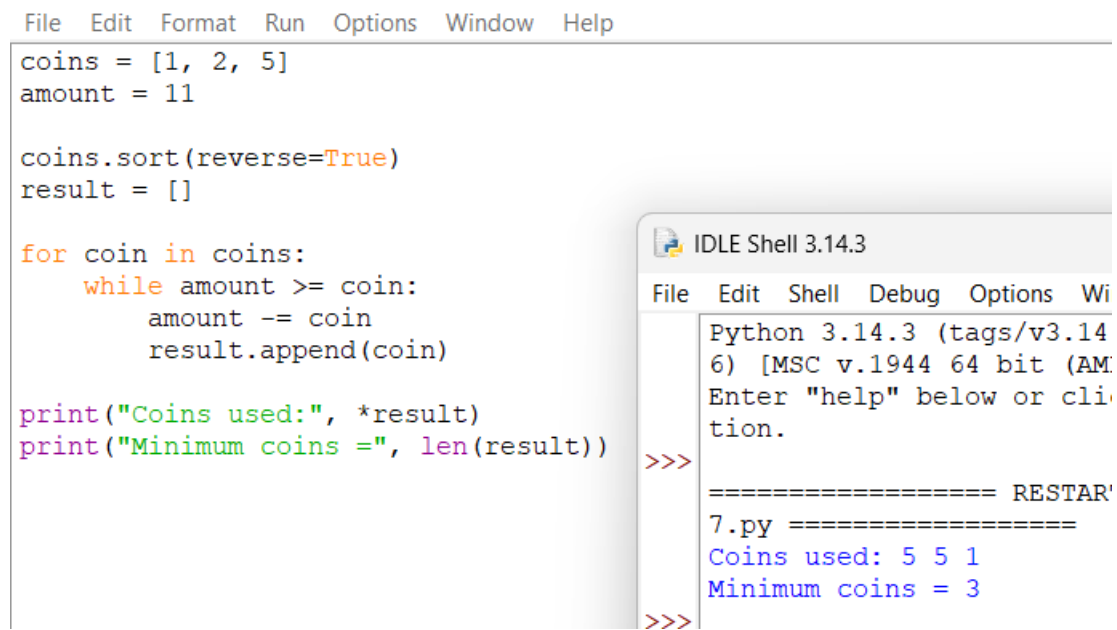
14. Coin Change – Vending Machine

Aim: To find the minimum number of coins required by a vending machine to provide the given change.

Algorithm

1. Read the denominations and amount.
2. Sort coins in descending order.
3. Select the largest possible coin.
4. Subtract it from the remaining amount.
5. Repeat until the amount becomes zero.
6. Display the coins and their count.

Execution:



```
File Edit Format Run Options Window Help
coins = [1, 2, 5]
amount = 11

coins.sort(reverse=True)
result = []

for coin in coins:
    while amount >= coin:
        amount -= coin
        result.append(coin)

print("Coins used:", *result)
print("Minimum coins =", len(result))
```

IDLE Shell 3.14.3

```
File Edit Shell Debug Options Wi
Python 3.14.3 (tags/v3.14.3:323c59a,
6) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" ab
tion.

>>>
===== RESTART: C:/Users/Thanushma/
7.py =====
Coins used: 5 5 1
Minimum coins = 3
>>>
```

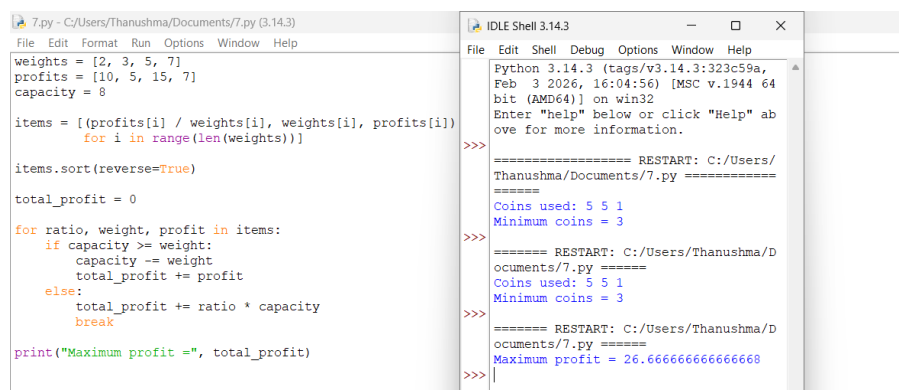
15. Fractional Knapsack – Warehouse Packing

Aim: To maximize the total value of items placed in a warehouse within a limited weight capacity using the Fractional Knapsack algorithm.

Algorithm

1. Calculate profit-to-weight ratio for every item.
2. Sort items in decreasing order of ratio.
3. Select the highest-ratio item first.
4. Add complete items while capacity permits.
5. Take a fraction of the next item when required.
6. Calculate and display the maximum profit.

Execution:



```
7.py - C:/Users/Thanushma/Documents/7.py (3.14.3)
File Edit Format Run Options Window Help
weights = [2, 3, 5, 7]
profits = [10, 5, 15, 7]
capacity = 8

items = [(profits[i] / weights[i], weights[i], profits[i])
         for i in range(len(weights))]

items.sort(reverse=True)

total_profit = 0

for ratio, weight, profit in items:
    if capacity >= weight:
        capacity -= weight
        total_profit += profit
    else:
        total_profit += ratio * capacity
        break

print("Maximum profit =", total_profit)
```

IDLE Shell 3.14.3

```
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Coins used: 5 5 1
Minimum coins = 3
>>>
===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Coins used: 5 5 1
Minimum coins = 3
>>>
===== RESTART: C:/Users/Thanushma/Documents/7.py =====
Maximum profit = 26.666666666666668
>>>
```

